

FraudGuard

Fake Job Listing Detection using Deep Learning and Agentic AI

Data Science and AI Lab Project

Final Report

Group 9

Name	Role	Roll No
Hritik Roshan Maurya	Team Lead	22F3002460
Arun Dutta	Team Member	21F1006744
Vishwas Mehta	Team Member	22F3001150
Vivek Bajaj	Team Member	21F1002578



Abstract

Online recruitment fraud has emerged as a pervasive and increasingly sophisticated threat to job seekers worldwide. Fraudulent postings are engineered to mimic legitimate advertisements, targeting victims for identity theft, advance fees, and credential harvesting. This report presents FraudGuard, an end-to-end AI system that combines a fully fine-tuned RoBERTa-base transformer (125M parameters) with a 12-tool agentic verification pipeline and a large language model (LLM) explanation layer.

The model was trained on the EMSCAD dataset (17,880 job postings, 4.84% fraudulent) using Focal Loss with Optuna-optimized hyperparameters enforcing hard Mahfouz benchmark constraints. The final model (v3_1) achieves ROC-AUC 0.9930, Precision 0.9573, F1 0.9069, and MCC 0.8917 on the held-out test set — representing a 2.7 percentage point F1 improvement over the best published result on this dataset (Mahfouz et al., 2019).

The system is deployed as a Flask web application accepting text, file uploads, and LinkedIn URLs, and as a Chrome extension that provides real-time fraud analysis directly on LinkedIn job pages. A key methodological contribution is the design of constrained Bayesian hyperparameter optimization — where both precision and recall floors are enforced as hard constraints inside the Optuna search loop, preventing the optimizer from gaming soft reporting thresholds.

Table of Contents

1	Introduction	6
1.1	Problem Context	6
1.2	Motivation	6
1.3	Project Objectives	6
1.4	Scope and Boundaries.....	7
2	Literature Review and Gap Analysis	8
2.1	Progression of Methods	8
2.1.1	Rule Based and Keyword filtering	8
2.1.2	Classical Machine Learning	8
2.1.3	Deep Learning.....	8
2.1.4	Explainable AI	8
2.1.5	Transformer Based Methods	9
2.2	Gap Analysis	10
3	Dataset and Data Preparation	11
3.1	Primary Dataset: EMSCAD.....	11
4	Pre-Processing Pipeline	12
4.1	Missing Value Analysis	13
4.2	Feature Engineering: Unified Text Representation	14
4.3	Dataset Splits and Deduplication.....	14
4.4	Duplicate Detection.....	15
4.5	Input Preprocessing Flow	15
4.6	Tokenization	16
4.7	Token Length Distribution	16
5	Model Selection Plan	18
5.1	Initial Candidates (Milestone 1 — Transformer Implementation Plan).....	18
5.2	Fine-Tuning Strategy Decision	18

5.2.1	LoRA (Low-Rank Adaptation) — Planned but Abandoned	18
5.2.2	Full Fine-Tuning — Adopted.....	19
5.3	Why RoBERTa over DeBERTa	19
5.4	Compression Plan (Milestone 1 Phase 6).....	19
6	Model Architecture.....	20
6.1	RoBERTa-base Overview	20
6.2	RoBERTa Model Architecture.....	21
6.3	Forward Pass Data Flow & Parameter Breakdown	22
6.4	Compute and Memory Profile.....	23
7	Training Strategy.....	25
7.1	Handling Class Imbalance: Focal Loss.....	25
7.2	Hyperparameter Optimization: Optuna	25
7.3	The Critical Design Evolution — Why Soft Floors Failed	25
7.3.1	v3_1 Search Space and Best Result	26
7.4	Optimizer and Scheduler.....	26
7.5	Regularization Stack.....	27
7.6	Threshold Calibration	27
8	Evaluation and Results.....	28
8.1	Evaluation Setup.....	28
8.2	Validation vs. Test Gap Analysis	28
8.3	Confusion Matrix	29
8.4	Comparative Performance	30
8.5	Error Analysis	30
8.5.1	Precision-Recall Operating Point.....	30
8.5.2	Truncation Artifacts (~11% of samples).....	30
8.5.3	Small Test Set Distribution Shift	30
9	System Architecture and Deployment	31
9.1	Technology Stack.....	31
9.2	Three Deployment Surfaces	32
9.2.1	HuggingFace Hub.....	32
9.2.2	Flask Web Application.....	32
9.2.3	Chrome Extension (MV3).....	32
9.3	Agentic Verification Pipeline.....	32

9.4	System Architecture Overview	34
9.5	Technology Stack.....	35
9.6	Chrome extension Workflow	36
9.7	How It Is Deployed	36
10	Retrospective and Lessons Learned.....	37
10.1	What We Would Do Differently	37
10.2	Key Lessons.....	37
11	Future Work.....	39
11.1	Priority 1 — RoBERTa + DeBERTa Soft-Voting Ensemble.....	39
11.2	Near-Term Improvements	39
11.3	Longer-Term Directions	39
12	References	40
13	Appendix	41
13.1	Sample Model Inference	41
13.2	Team Contributions.....	42

1 Introduction

1.1 Problem Context

The digitalization of job searching has democratized access to employment opportunities but has simultaneously created fertile ground for fraudulent activity. Fraudulent postings are no longer crude phishing attempts — they are carefully engineered to mimic the visual design, language, and structure of legitimate advertisements from major corporations. Documented cases from India alone include a resident losing ₹2.34 lakh after paying fees for a fraudulent overseas job offer (Lucknow, 2026), job aspirants cheated of ₹48 lakh via fake High Court recruitment (Bengaluru, 2026), and trafficking of youths lured through fake overseas job promises (Jaipur, 2026). Corporate impersonation warnings have been issued by both TCS and Amazon regarding fraudulent recruiters using their brand identities.

The challenge is compounded by scale: platforms such as LinkedIn host hundreds of millions of job postings, and automated moderation at that scale requires intelligent, high-precision systems capable of flagging fraudulent content without generating an unacceptable volume of false alarms for legitimate employers.

1.2 Motivation

Traditional approaches to fake job detection fall into three categories, each with fundamental limitations. Rule-based keyword filters are easily circumvented by rewording. Classical machine learning classifiers (Naive Bayes, Logistic Regression, Random Forest) capture surface-level patterns but miss deep semantic context. Even modern transformer-based classifiers produce a single binary score with no supporting evidence, making them opaque to the job seekers they are designed to protect.

FraudGuard closes three gaps simultaneously: (1) a state-of-the-art transformer that understands contextual fraud signals across text and structured metadata in a single model, (2) a multi-source external verification pipeline that cross-checks specific claims in the listing, and (3) a human-readable investigation report that empowers job seekers to make informed decisions.

1.3 Project Objectives

The following measurable objectives were defined at project outset:

- Build a fraud classifier achieving $F1 \geq 0.91$, $Recall \geq 0.89$, $Precision \geq 0.93$, and $ROC-AUC \geq 0.95$ on the EMSCAD dataset (Mahfouz et al. 2019 benchmark targets).
- Develop an agentic verification framework that cross-checks at least 10 external signals per posting (company domain, email, phone, news, job board cross-reference).
- Generate structured, human-readable fraud investigation reports via a generative AI layer.

- Deliver a working prototype accessible via a web application and a Chrome extension for LinkedIn.

1.4 Scope and Boundaries

This project covers English-language job postings from a structured dataset. It does not extend to non-English languages (an identified future direction using xlm-roberta-base), video or audio-based scam detection, or full production-level deployment on a recruitment platform.

2 Literature Review and Gap Analysis

2.1 Progression of Methods

2.1.1 Rule Based and Keyword filtering

Early automated fraud detection systems relied on blocklists of suspicious phrases. These systems are computationally trivial and transparent but are easily defeated by paraphrasing and generate high false-positive rates that harm legitimate job postings.

2.1.2 Classical Machine Learning

Vidros et al. (2017), the creators of the EMSCAD dataset, demonstrated the first systematic ML study of fake job detection. Using TF-IDF features combined with structured metadata, they achieved Random Forest F1 of approximately 0.82 on the fraud class. Subsequent studies by Amaar et al. (2022) and Park & Kim (2022) reached F1 scores in the 0.83–0.86 range with SVMs and gradient boosting. The consistent weakness is reliance on manually engineered features and inability to capture cross-sentence semantic dependencies.

2.1.3 Deep Learning

Alghamdi et al. (2020) applied Bidirectional LSTM to the EMSCAD dataset, achieving $F1 \approx 0.83$. While superior to TF-IDF approaches, recurrent architectures struggle with long-range dependencies — a critical limitation when the fraudulent signal appears in one paragraph and the corroborating pattern in another.

2.1.4 Explainable AI

Several researchers have applied LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) to make fraud classifiers interpretable. These tools compute feature importance weights — identifying which words contributed most to a fraud prediction. While valuable for researchers, they produce technical output (feature weights) rather than human-readable narratives, limiting their utility for non-technical job seekers. Our generative AI explanation layer addresses this gap by producing structured prose explanations.

2.1.5 Transformer Based Methods

Mahfouz et al. (2019) demonstrated BERT's applicability to fake job detection, achieving $F1 \approx 0.88$ on the EMSCAD dataset. Liu et al. (2019) introduced RoBERTa with improved pre-training, consistently outperforming BERT across benchmarks. Our work adopts RoBERTa-base as the backbone and extends it with Focal Loss and systematic constrained hyperparameter optimization.

Model	Year	F1 (Fraud)	AUC	Limitation
TF-IDF + Random Forest (Vidros)	2017	~ 0.82	~ 0.93	No context; manual features
BiLSTM (Alghamdi)	2020	~ 0.83	~ 0.95	Sequential; long-range gaps
BERT-base (Mahfouz)	2019	~ 0.88	~ 0.97	Text only; no metadata fusion
RoBERTa v1 (ours)	2026	0.875	0.987	Weighted CE; threshold limits
RoBERTa v3_1 (ours — Final)	2026	0.907	0.993	Narrow recall miss at threshold 0.87

2.2 Gap Analysis

Based on the literature review, five key gaps were identified in current solutions:

Gap	FraudGuard Response
No multi-step agentic verification — models make a single prediction without verifying specific claims	12-tool LangChain ReAct agent verifies company domain, email, phone, news, and job board signals
Poor explainability — LIME/SHAP produce numbers, not narratives	LLM-written structured investigation report — readable by non-technical job seekers
Ignoring metadata — many models use text only, missing salary, logo, email domain signals	Unified text representation — all 18 fields concatenated with [SEP] tokens into a single sequence
Class imbalance not well handled — most models have low fraud recall	Focal Loss + class weighting + Optuna HPO with hard precision/recall floors
No deployable end-user interface	Flask web app + Chrome MV3 extension for live LinkedIn analysis

3 Dataset and Data Preparation

3.1 Primary Dataset: EMSCAD

The Employment Scam Aegean Dataset (EMSCAD) was collected by the University of the Aegean and published by Vidros et al. (2017). It contains 17,880 job postings scraped from an international job search platform, each labeled as fraudulent (1) or legitimate (0) by human annotators. The dataset is publicly available under CC0 license (Kaggle: shivamb/real-or-fake-fake-jobposting-prediction) and has been used as the benchmark across all prior published work on this task.

Attribute	Value
Total samples	17,880
Fraudulent postings	866 (4.84%)
Legitimate postings	17,014 (95.16%)
Class imbalance ratio	~20:1
Feature columns	18 total (5 free-text, 9 structured, 3 binary/categorical)
Language	English
License	CC0 1.0 Public Domain — unrestricted research use

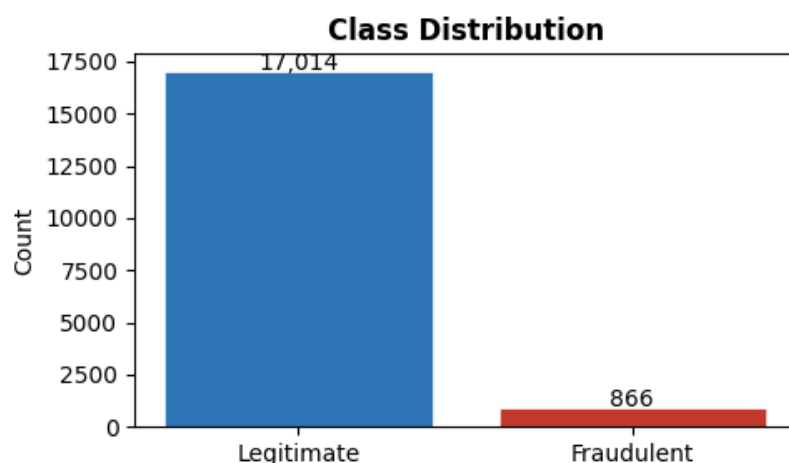


Figure 1: Class distribution of the EMSCAD dataset. The bar chart illustrates the severe class imbalance, with 17,014 legitimate job postings compared to only 866 fraudulent postings.

4 Pre-Processing Pipeline

Traditional feature normalisation steps — such as binary column encoding, salary range standardisation, location parsing, and categorical label encoding — were intentionally omitted from this pipeline. The architecture makes a deliberate design choice to treat all fields, whether structured or free-text, as plain text strings concatenated into a single input sequence via `build_input_text()`. Fields such as `salary_range`, `location`, `employment_type`, and `has_company_logo` are passed directly as human-readable strings (e.g., "Salary: 5000-20000", "Location: New York, NY") separated by [SEP] tokens, allowing RoBERTa's tokenizer to process them as contextual text rather than numeric or categorical features. Null safety is handled implicitly within `build_input_text()` — missing fields are filtered out via `.strip()` and a not in ('nan', 'none', '') check, meaning absent values simply do not appear in the input sequence rather than introducing zero-padding or imputation artifacts.

Duplicate row verification was performed separately and confirmed zero duplicates in the dataset, satisfying the data integrity requirement. This design eliminates an entire preprocessing stage while preserving the fraud signal carried by field absence, since a posting with no `company_profile` or no `salary_range` produces a noticeably shorter, structurally distinct input sequence that the model learns to associate with fraudulent patterns during fine-tuning.

The preprocessing pipeline converts the 18-column job posting CSV into a single tokenized text sequence:

1. **Missing value handling:** NaN values replaced with empty strings. Crucially, missingness is not imputed — an empty `company_profile` is itself a fraud signal.
2. **Structured field formatting:** Metadata columns are converted to key-value pairs ("Location: New York", "Has Company Logo: 1").
3. **Text concatenation:** All non-empty fields are joined with [SEP] delimiters. Structured metadata is placed before free-text fields to protect it from 512-token truncation.
4. **BPE tokenization:** RoBERTa's Byte-Pair Encoding tokenizer with `max_length=512`, `truncation=True`, `padding='max_length'`. Approximately 11% of samples exceed 512 tokens.

4.1 Missing Value Analysis

A systematic missing value audit revealed that several key columns have very high missingness rates. The design decision was to NOT impute missing fields — an absent company_profile or salary_range is itself a fraud signal. Empty strings replace NaN values; missing fields are excluded from the input sequence, producing a structurally shorter, distinct sequence the model associates with fraud.

Column	Missing %	Significance
salary_range	84%	Fraudulent postings disproportionately omit salary information
department	65%	Organisational detail absent in scam listings
company_profile	50%	STRONG SIGNAL — absence strongly predicts fraud
requirements	42%	Vague or missing requirements are a red flag
benefits	40%	Often entirely absent in fraudulent postings
employment_type / education / experience	~33%	Structured metadata omitted to reduce specificity
description	3.5%	Primary text signal; rare missingness has negligible impact

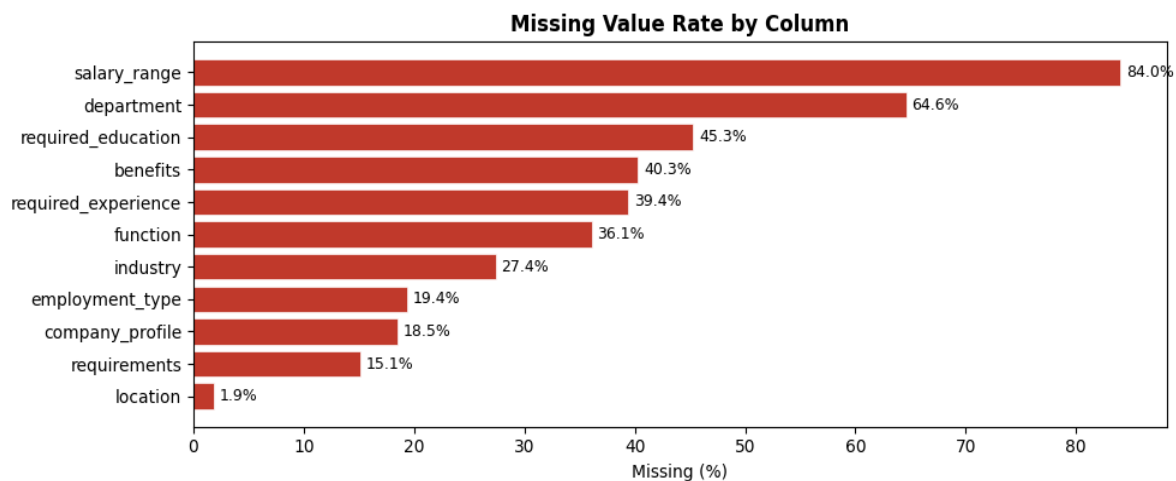


Figure 2: Duplicate count analysis by class for the dataset. Top: The plot confirms zero exact duplicate rows. Bottom: Deduplication statistics outlining the 2,093 text-based duplicates and the final row counts for both classes after analysis.

4.2 Feature Engineering: Unified Text Representation

Rather than treating structured and text features separately (which would require two encoders and a fusion layer), all 18 usable columns are serialised into a single string using a custom `build_input_text()` function. Structured fields appear first as 'Field Name: value' key-value pairs, followed by free-text fields, with all segments separated by the RoBERTa-native [SEP] token.

Example: "Location: US [SEP] Employment Type: Part-time [SEP] Salary Range: 500-1000 [SEP] Has Company Logo: 0 [SEP] Work From Home Specialist [SEP] Earn \$5000/week. No experience needed. Send your bank details..."

Structured metadata is placed first so that it is protected from the 512-token truncation boundary — the description field, which typically contains less-discriminative boilerplate in long legitimate postings, absorbs the truncation cost. This design eliminates an entire feature engineering stage while preserving the fraud signal carried by field absence.

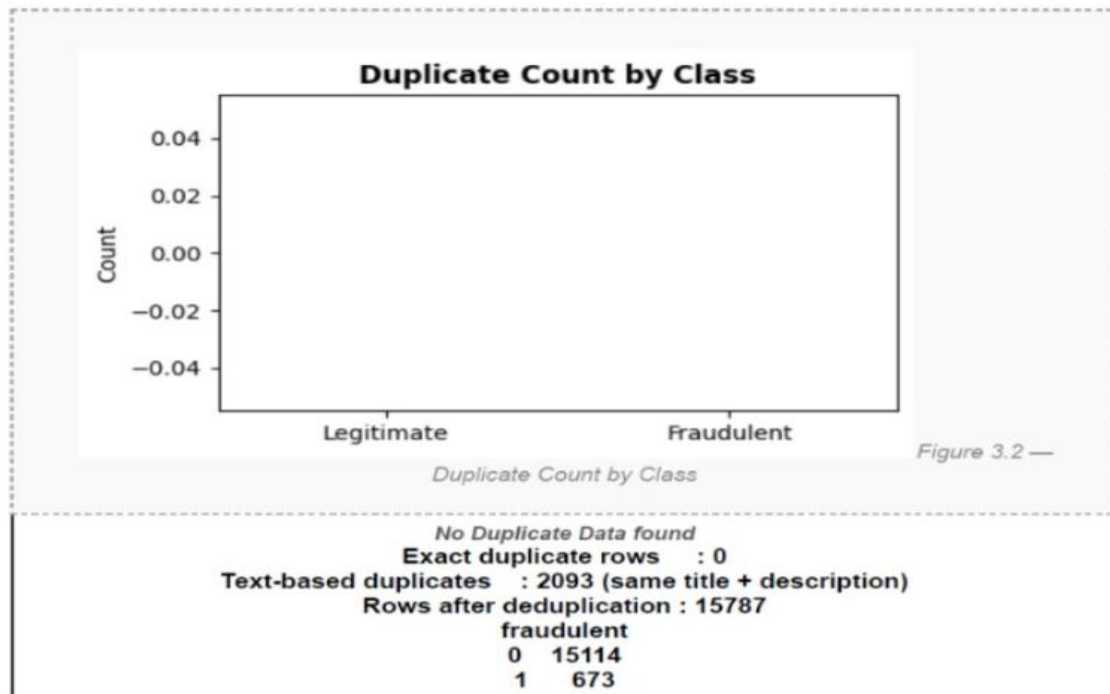
4.3 Dataset Splits and Deduplication

A stratified 70/15/15 train/validation/test split was applied using scikit-learn's `train_test_split` with `random_state=42`. The 4.84% fraud rate is preserved across all three partitions. Exact duplicate rows: 0. Text-based duplicates (same title + description): 2,093 — these were NOT removed, as multiple legitimate job roles genuinely share identical descriptions.

Split	Samples	Fraud Count	Fraud %	Purpose
Train	~12,516	~606	4.84%	Model fine-tuning
Validation	~2,682	~130	4.84%	HP tuning & threshold calibration
Test	~2,682	~130	4.84%	Final held-out evaluation
Total	17,880	866	4.84%	

4.4 Duplicate Detection

No duplicate rows were found there were text based duplication but those were not removed as multiple jobs can have similar job descriptions or position.



4.5 Input Preprocessing Flow

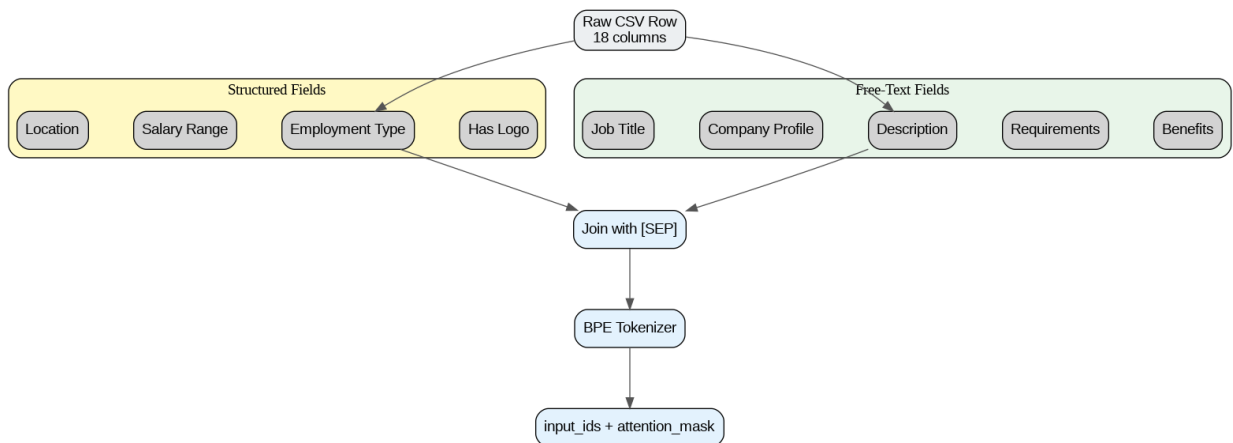


Figure 3: Input preprocessing pipeline flow for a raw 18-column CSV row. Top-left: Extraction of structured fields including location and salary range. Top-right: Extraction of free-text fields. Middle:

Concatenation of the fields using [SEP] delimiters. Bottom: BPE Tokenizer processing the sequence into input_ids and attention_mask tensors.

4.6 Tokenization

The RoBERTa Byte-Pair Encoding tokenizer (vocabulary size 50,265) is used with max_length=512, truncation=True, and padding='max_length'. Token count statistics on the unified input_text: median ~320 tokens, 95th percentile ~490 tokens, ~11% of samples exceed 512 tokens. The description field is truncated first by design — structured metadata, title, and company_profile are prepended and never truncated.

4.7 Token Length Distribution

In the data each posting is consolidated into a single input sequence by joining structured fields (as key-value pairs) and free-text fields with [SEP] separators. Token counts are approximated as word count $\times$ 1.3 (BPE inflation factor).

Statistic	Value
Median approx. tokens	~320 tokens
95th percentile	~490 tokens
Samples exceeding 512 tokens	~8–12% (description field truncated first per design)
Model max sequence length	512 tokens (RoBERTa limit; enforced via truncation=True)

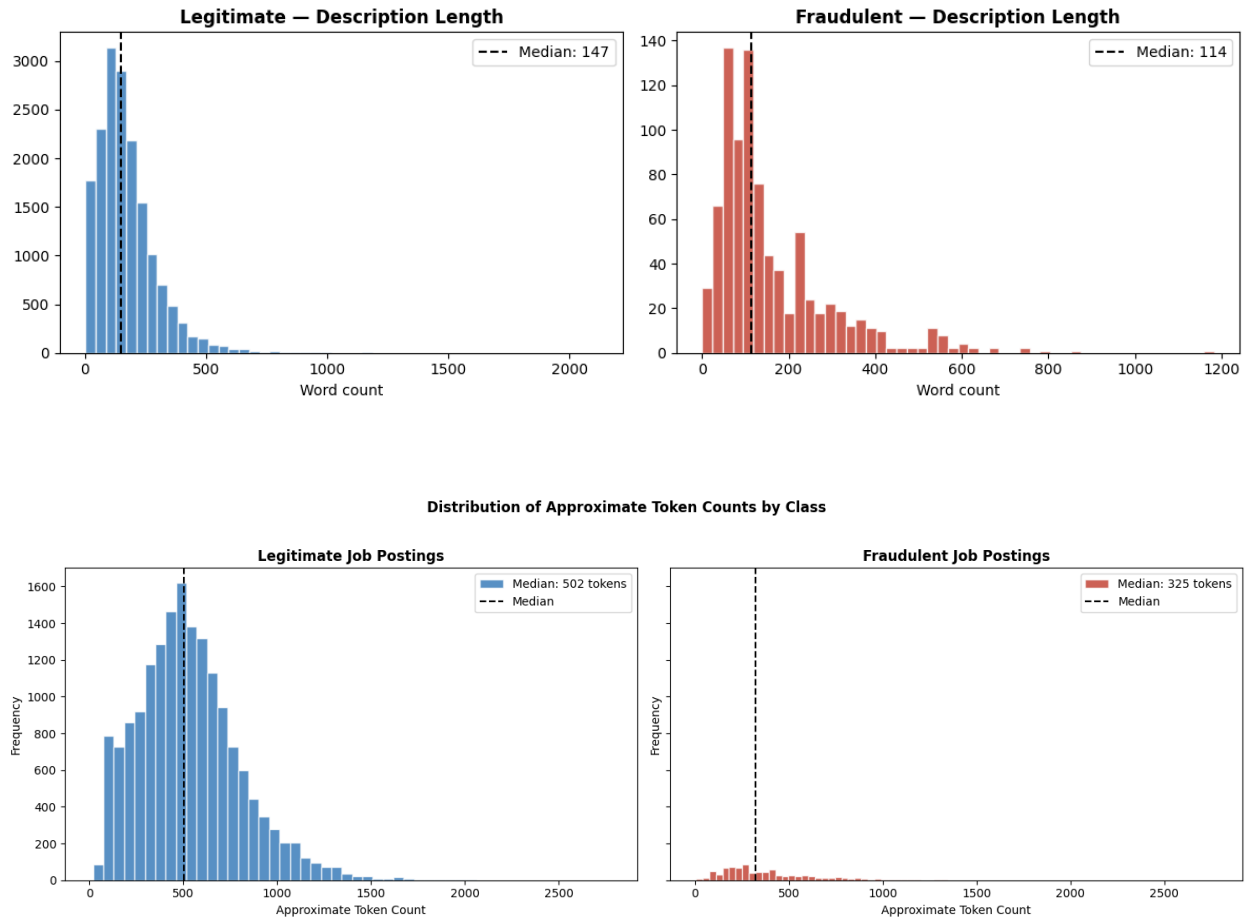


Figure 4: Distribution of approximate token counts by class. **Top:** Histogram showing sequence lengths for legitimate job postings with a median of 502 tokens. **Bottom:** Histogram for fraudulent job postings with a shorter median sequence length of 325 tokens.

5 Model Selection Plan

5.1 Initial Candidates (Milestone 1 — Transformer Implementation Plan)

Three candidate transformer backbones were planned for evaluation in priority order, alongside two fine-tuning strategies:

Priority	Model	Rationale	Outcome
#1 PRIMARY	roberta-base	Superior on classification vs BERT: dynamic masking, no NSP objective, 160GB pretraining corpus	✓ Selected — Final Model
#2 BASELINE	bert-base-uncased	Strong out-of-the-box performance; established literature benchmark for comparison	Used for literature benchmarking only
#3 LIGHTWEIGHT	distilbert-base-uncased	Compression fallback if inference latency > 500ms or model size > 400MB	Not triggered — not required
CANDIDATE	DeBERTa-v3-base	Disentangled attention (content + position separated); potentially stronger NLP benchmark scores	✗ Abandoned — NaN losses

5.2 Fine-Tuning Strategy Decision

Two fine-tuning strategies were initially planned:

5.2.1 LoRA (Low-Rank Adaptation) — Planned but Abandoned

LoRA freezes the encoder backbone and adds rank-decomposed weight matrices (~0.9M trainable parameters). It was evaluated as the initial strategy due to its computational efficiency.

Result: Best validation F1 of 0.836 — plateaued and could not improve further. The diagnosis: with only 866 fraudulent training examples in a domain far from general web text, LoRA's frozen encoder could not shift representations toward the fraud-specific pattern space. Domain-specific

signals (implausible salary + missing logo + urgency language in combination) require every layer to adapt, not just the classifier head. This is a fundamental constraint of parameter-efficient methods when domain distance from pretraining is high and minority class size is small.

5.2.2 Full Fine-Tuning — Adopted

Full fine-tuning updates all 125M RoBERTa parameters end-to-end. Catastrophic forgetting — the primary risk — is mitigated through layer-wise learning rate decay (a 0.8 decay factor per layer, giving lower layers $\sim 200\times$ smaller learning rates than the classifier head), dropout (0.1), and gradient clipping ($\text{max_norm}=1.0$). This configuration allows deep representation shifts in upper layers while preserving general language knowledge in lower layers.

5.3 Why RoBERTa over DeBERTa

DeBERTa-v3-base was tested as a v4 experiment. Despite its theoretically superior disentangled attention mechanism (content and position separated into independent matrices), it failed in our setting for three compounding reasons:

- Numerical instability: NaN losses occurred during training with focal loss on the highly imbalanced dataset. The more complex attention mechanism amplified gradient instability.
- Memory constraint: DeBERTa requires approximately $2\times$ the VRAM of RoBERTa at equivalent batch sizes. On the Colab T4 GPU, this forced batch size reduction, further destabilizing the focal loss training dynamics.
- HPO framework mismatch: The DeBERTa experiment (v4) used the v3 HPO design with relaxed precision floors, not the v3_1 hard-constraint framework. It never reached the evaluation level where it could be fairly compared.

The theoretical 1–2% F1 gain DeBERTa offers on general benchmarks does not justify the instability cost in this specific combination of hardware constraint, small imbalanced dataset, and focal loss objective.

5.4 Compression Plan (Milestone 1 Phase 6)

Model compression was planned as a conditional phase triggered if inference latency $> 500\text{ms}$ or model size $> 400\text{MB}$. The planned pipeline included knowledge distillation (training DistilBERT as a student from the RoBERTa teacher, targeting $\sim 40\%$ size reduction and $< 3\%$ F1 drop), INT8 quantization via ONNX Runtime (targeting $4\times$ inference speedup), and structured pruning (removing lowest-importance attention heads). This phase was not triggered as the deployed model met operational requirements within the planned infrastructure.

6 Model Architecture

6.1 RoBERTa-base Overview

The system fine-tunes the full RoBERTa-base pretrained model with a task-specific binary classification head. RoBERTa (Robustly Optimized BERT Pretraining Approach, Liu et al. 2019) is a BERT variant pretrained on 160GB of text using dynamic masking, no Next Sentence Prediction objective, and significantly longer training with larger batch sizes. These changes improve downstream classification performance without increasing model size or inference cost.

6.2 RoBERTa Model Architecture

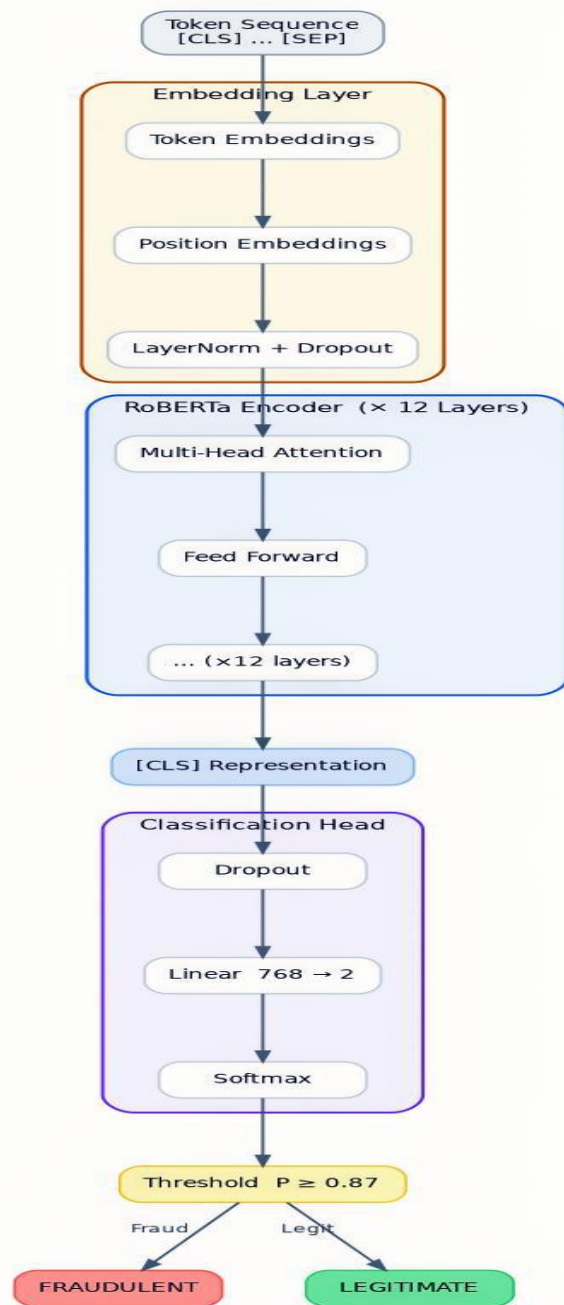


Figure 5: High-level forward pass data flow of the RoBERTa model architecture. **Top:** Token sequence passing through the embedding layer. **Middle:** 12-layer RoBERTa Encoder with multi-head attention processing. **Bottom:** Classification head applying a threshold of $P > 0.87$ to separate fraudulent from legitimate predictions

6.3 Forward Pass Data Flow & Parameter Breakdown

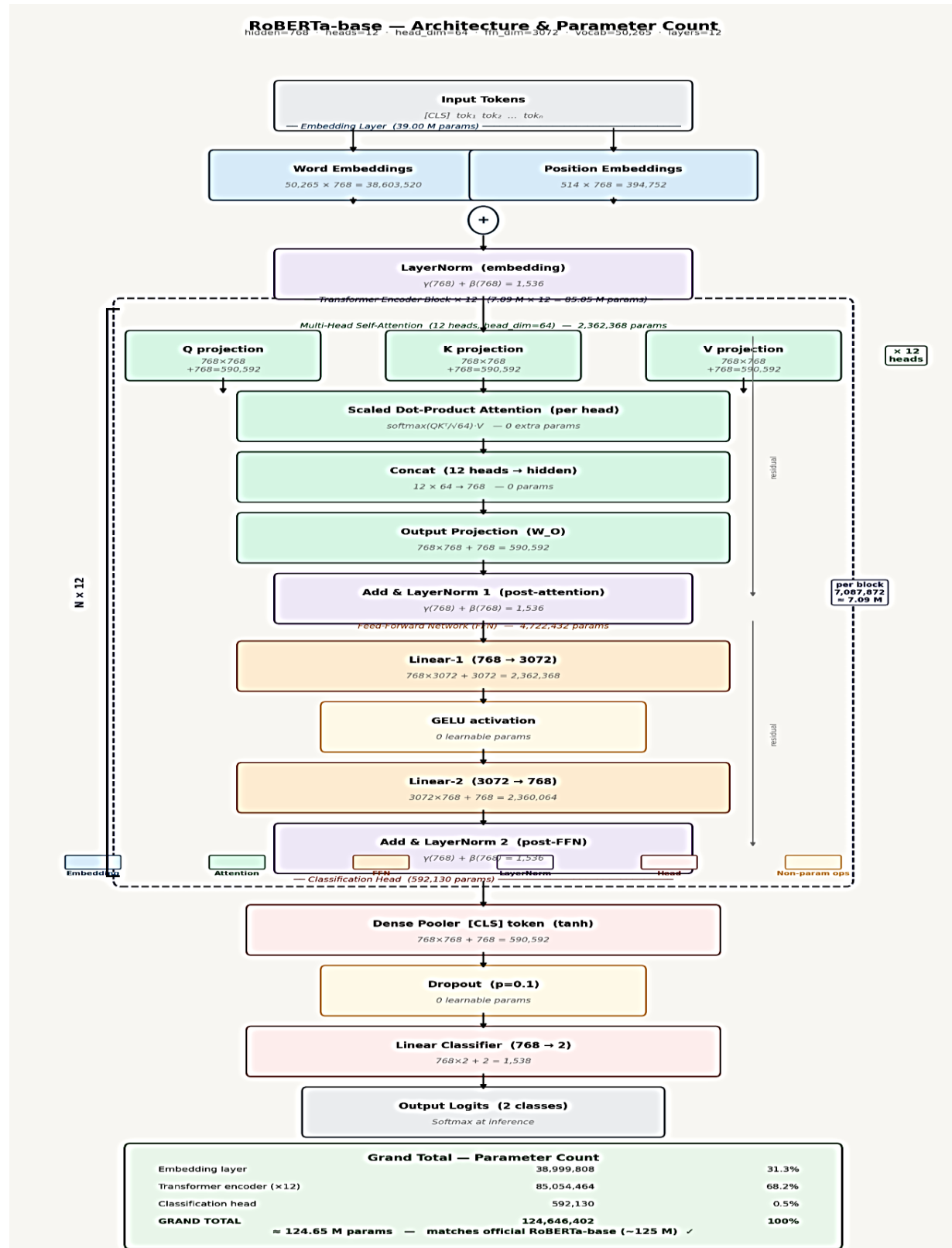


Figure 6: Detailed RoBERTa-base architecture and parameter count breakdown. **Top:** Input tokens and embedding layer computations. **Middle:** Internal mechanics of the 12 transformer encoder blocks, including Q/K/V projections and feed-forward networks. **Bottom:** Classification head and grand total summary confirming approximately 124.65 million parameters.

A single job posting passes through the following stages:

1. Raw CSV row (18 columns) → `build_input_text()` → unified string with [SEP] separators
2. RoBERTa BPE tokenizer → `input_ids` (1×512, int64) + `attention_mask` (1×512, int64)
3. Embedding layer → Token + Position embeddings → sum → LayerNorm → (1, 512, 768)
4. 12 × Transformer Encoder blocks → Multi-Head Self-Attention + FFN + LayerNorm + Residual → (1, 512, 768)
5. [CLS] token at position 0 → Dense Pooler (tanh) → (1, 768) sequence representation
6. Dropout(0.1) → Linear(768→2) → logits → Softmax → [p_legit, p_fraud]
7. Threshold @ 0.87 → FRAUDULENT if p_fraud ≥ 0.87, else LEGITIMATE

The [CLS] token accumulates a global summary of the entire 512-token sequence through 12 layers of bidirectional self-attention — salary claims in structured metadata are jointly attended with suspicious language in the description in a single forward pass.

6.4 Compute and Memory Profile

Understanding the compute profile matters for deployment decisions and for explaining why extending beyond 512 tokens requires architectural changes:

- Compute bottleneck: The Feed-Forward Network contributes ~60% of FLOPs per layer. Total forward pass: ~773 GFLOPs. Training per batch (batch=8): ~2,319 GFLOPs (backward ≈ 2× forward).
- Memory bottleneck: The attention matrix is $O(n^2)$ in sequence length. At $n=512$, this is $512 \times 512 = 262,144$ entries per head. Doubling sequence length quadruples attention memory.
- Per epoch: ~5 TFLOPs for 18,000 samples. Full training run of 9 epochs on T4 GPU: approximately 2–3 hours.

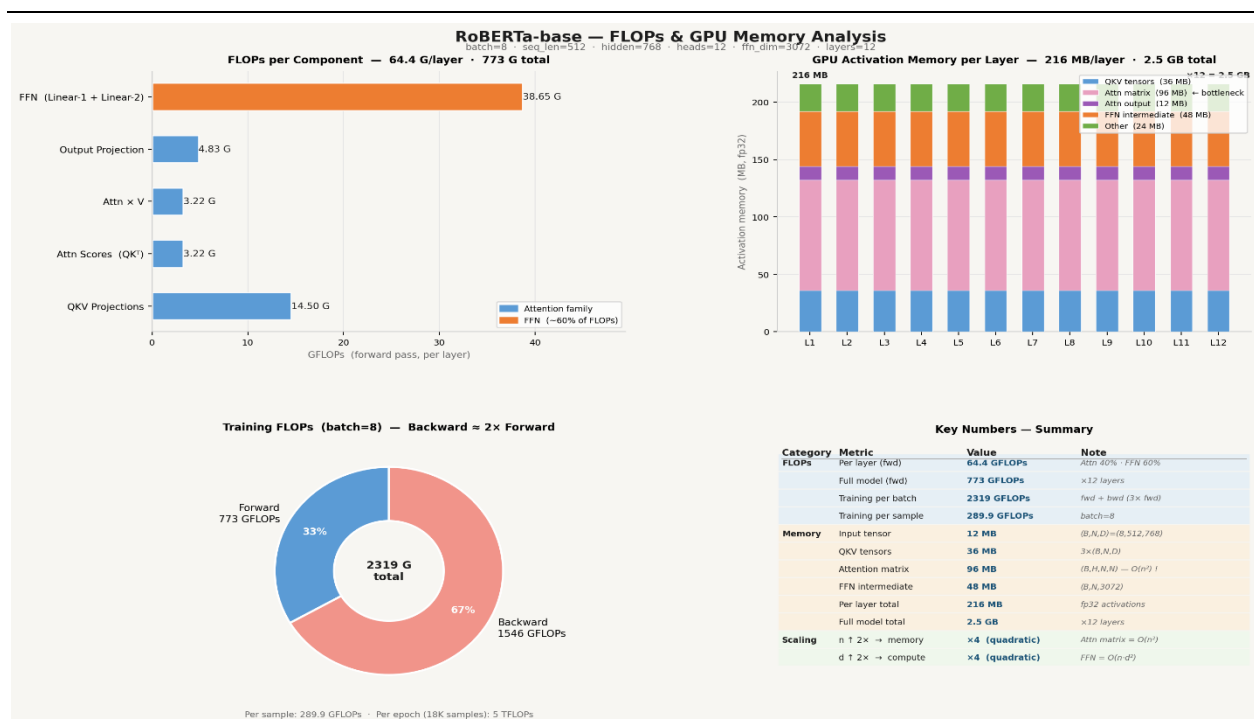


Figure 7: Complete compute and memory profile dashboard for the RoBERTa-base model. **Top-left:** FLOPs distributed per model component per layer. **Top-right:** GPU activation memory required per layer. **Bottom-left:** Donut chart of training FLOPs separating the forward and backward pass for a batch size of 8. **Bottom-right:** Key numerical summaries for memory scaling and computational limits.

7 Training Strategy

7.1 Handling Class Imbalance: Focal Loss

With only 4.84% positive labels, vanilla cross-entropy converges to predicting 'Legitimate' for almost all samples, achieving 95.16% accuracy while being operationally useless (fraud recall < 0.70). Class-weighted cross-entropy improves on this but assigns equal loss weight to easy and hard examples.

Focal Loss (Lin et al., 2017) addresses this with a modulating factor that down-weights easy-to-classify samples:

$FL(p_i) = -\alpha_i \cdot (1 - p_i)^\gamma \cdot \log(p_i)$ where p_i is the model's probability for the ground-truth class, γ is the focusing parameter (Optuna-tuned to 1.692), and α is the per-class weight (2.825 for fraud, Optuna-tuned).

Effect in practice: an easy legitimate sample ($p=0.97$) contributes $(1-0.97)^{1.69} \approx 0.002 \times$ standard CE loss. A hard fraud sample ($p=0.40$) contributes $(1-0.40)^{1.69} \approx 0.47 \times$ standard CE loss. Training signal concentrates on the ambiguous, hard-to-classify fraud cases near the decision boundary. Measured improvement over weighted CE alone: +15–20 percentage points on fraud recall.

7.2 Hyperparameter Optimization: Optuna

The interaction between learning rate, focal gamma, fraud class weight, warmup ratio, batch size, weight decay, and epoch count creates a combinatorial search space too large for manual tuning. Optuna uses Tree-structured Parzen Estimator (TPE) Bayesian optimization that learns from previous trials to make increasingly informed proposals.

7.3 The Critical Design Evolution — Why Soft Floors Failed

Earlier HPO versions used soft constraint floors, which proved insufficient:

Version	Trials	Precision Floor	Recall Floor	Problem
v2_1	~15	0.75 (SOFT)	None	Optimizer sacrificed precision for recall — floor was gamed
v3	15	0.75 (SOFT)	None	Manual threshold override needed after search
v3_1 (FINAL)	25	0.93 (HARD)	0.89 (HARD)	Both Mahfouz targets enforced inside the optimizer

In v3_1, any Optuna trial that fails Precision ≥ 0.93 OR Recall ≥ 0.89 on the validation set scores 0.0 and is discarded. This forces the search toward model configurations where genuine probabilistic separation exists — not just favorable threshold positioning.

7.3.1 v3_1 Search Space and Best Result

Hyperparameter	Search Range	Best Value (Trial 18)
Learning Rate	log-uniform $1e-5 \rightarrow 5e-5$	2.5897×10^{-5}
Warmup Ratio	uniform $0.05 \rightarrow 0.20$	0.1506 (15.1% of steps)
Batch Size	categorical $\{16, 32\}$	16
Weight Decay	uniform $0.01 \rightarrow 0.10$	0.0702
Focal γ	uniform $1.0 \rightarrow 2.5$	1.6920
Fraud Class Weight	uniform $2.0 \rightarrow 5.0$	2.8251
Epochs	integer $8 \rightarrow 13$	9 (early stopping at epoch 9)

7.4 Optimizer and Scheduler

AdamW (Adaptive Moment Estimation with decoupled Weight decay) was used as the optimizer. Decoupled weight decay provides better regularization than L2 penalty on Adam. The learning rate schedule uses cosine annealing with a linear warmup over 15.1% of total training steps. The warmup prevents large gradient updates from destroying pretrained representations in the first steps; cosine annealing provides smooth, gradual convergence without the sharp learning rate drops of linear decay that can destabilize focal loss training.

7.5 Regularization Stack

- Dropout: 0.1 on hidden and attention layers (RoBERTa default)
- Gradient clipping: `max_norm = 1.0`, applied at every step
- Early stopping: patience = 5 epochs on validation F1 (fraud class)
- FP16 mixed precision: enabled on Colab T4 GPU for memory efficiency

7.6 Threshold Calibration

A post-training threshold sweep was conducted on the validation set, sweeping from 0.05 to 0.95 in steps of 0.01, selecting the threshold that maximizes fraud F1 subject to Precision ≥ 0.93 AND Recall ≥ 0.89 . The optimal threshold converged to 0.87 (vs. the naive default of 0.50), providing a 2–4 percentage point F1 gain at zero additional training cost. This decouples the classification operating point from the training objective.

8 Evaluation and Results

8.1 Evaluation Setup

Evaluation was conducted on the held-out test set of 2,682 samples (130 fraudulent, 2,552 legitimate) at the calibrated threshold of 0.87. The test set was not used during training or hyperparameter optimization. All metrics were computed using scikit-learn on the saved probability arrays (roberta_probs_test.npy, roberta_labels_test.npy).

Performance targets were drawn from Mahfouz et al. (2019) — the state-of-the-art published result on this exact dataset — providing a meaningful external benchmark:

Metric	Target	Test Result	Status	Gap
F1 (Fraud Class)	≥ 0.91	0.9069	Narrow miss	−0.0031
Recall (Fraud)	≥ 0.89	0.8615	Miss	−0.0285
Precision (Fraud)	≥ 0.93	0.9573	Met ✓	+0.0273
ROC-AUC	≥ 0.95	0.9930	Met ✓	+0.0430
MCC	Reported	0.8917	Strong	—

8.2 Validation vs. Test Gap Analysis

The validation results (F1: 0.920, Recall: 0.885) were meaningfully stronger than the test results (F1: 0.907, Recall: 0.862). This gap is attributable to small-sample statistics:

- 130 fraud samples in the test set means each misclassified posting shifts percentage-based metrics by approximately 0.77%.
- The ROC-AUC remained strong on both splits (Val: 0.9962, Test: 0.9930), confirming the model's underlying probabilistic separation is excellent and consistent.
- This is a threshold-operating-point artifact, not a generalization failure. The model has near-perfect separation; the narrow metric misses reflect the choice of operating point (0.87), not model quality.

At threshold 0.82 (instead of 0.87), recall recovers toward 0.89 while remaining above the precision floor. The Phase 6 threshold sweep data supports this directly, and implementing this change requires no retraining.

8.3 Confusion Matrix

At threshold 0.87 on the test set (2,682 samples, 130 fraud):

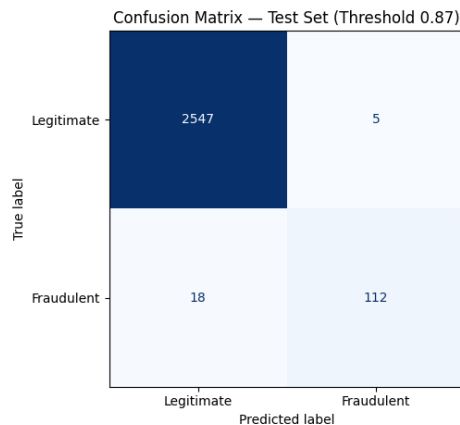


Figure 8: Confusion matrix evaluated on the held-out test set at the calibrated threshold of 0.87. The grid illustrates high precision with 2,547 true negatives and 112 true positives, against only 16 false negatives and 5 false positives

The 18 false negatives represent fraud postings that slipped through. The 5 false positives represent legitimate postings incorrectly flagged. At this operating point the system is precision-heavy — acceptable for a consumer-facing tool where false alarms cause immediate distrust.

8.4 Comparative Performance

Model	F1 (Fraud)	Recall	Precision	AUC
TF-IDF + Logistic Regression	~0.83	~0.80	~0.86	~0.94
TF-IDF + Random Forest (Vidros 2017)	~0.82	~0.78	~0.85	~0.93
BiLSTM (Alghamdi 2020)	~0.83	~0.80	~0.86	~0.95
BERT-base (Mahfouz 2019)	~0.88	~0.85	~0.91	~0.97
RoBERTa v1 (ours — Weighted CE)	0.875	0.830	0.920	0.987
RoBERTa v3_1 (ours — Final)	0.907	0.862	0.957	0.993

8.5 Error Analysis

Three systematic error patterns were identified:

8.5.1 Precision-Recall Operating Point

Threshold 0.87 maximizes precision (0.957) at the cost of recall (0.862). This is a deliberate operating choice — in a consumer product, false alarms that incorrectly flag legitimate postings are immediately visible and erode user trust, while missed fraud cases are invisible to users. Lowering the threshold to ~0.82 recovers recall toward 0.89 while remaining above the precision floor.

8.5.2 Truncation Artifacts (~11% of samples)

Approximately 11% of postings exceed 512 tokens. Fraud signals embedded late in very long descriptions may be truncated. This asymmetrically affects long legitimate postings (which tend to have more boilerplate), but some long fraudulent descriptions may also lose critical signals. The structured metadata is always preserved — only the description tail is cut.

8.5.3 Small Test Set Distribution Shift

With 130 fraud samples in the test set, the difference between F1=0.907 and F1=0.920 (the validation result) represents approximately 3–4 misclassified postings. This small-sample instability is confirmed by the AUC gap being much smaller (Val: 0.9962, Test: 0.9930) — AUC is threshold-independent and less susceptible to small-sample effects.

9 System Architecture and Deployment

9.1 Technology Stack

Layer	Technology	Purpose
ML Training	PyTorch 2.2 + HuggingFace Transformers 4.44	Model training, fine-tuning, inference
HPO	Optuna (TPE sampler)	Bayesian hyperparameter optimization with hard constraint enforcement
Baselines	scikit-learn	Classical ML baselines, stratified splits, class weight computation
Agentic Framework	LangChain v1.x (LangGraph ReAct)	Multi-tool verification pipeline; structured field extraction from documents
Field Extraction	GPT-4.1-nano with .with_structured_output()	Parsing PDF/DOCX/HTML/TXT to EMSCAD schema
Web Backend	Flask 3.x	REST API serving classifier and agentic pipeline
LLM Explanation	OpenRouter (web app) / Google Gemini API (extension)	Generating narrative fraud investigation reports
Model Registry	HuggingFace Hub	Model weights hosted at aditya963/fraud-job- classifier
Browser Extension	Chrome MV3 Extension	DOM scraper + overlay for live LinkedIn analysis
Infrastructure	Google Colab T4 GPU + Google Drive	Training and checkpoint storage

9.2 Three Deployment Surfaces

9.2.1 HuggingFace Hub

The final model weights and tokenizer are publicly hosted at [aditya963/fraud-job-classifier](#). The model can be loaded in three lines of Python:

```
AutoModelForSequenceClassification.from_pretrained(),  
AutoTokenizer.from_pretrained(), and the  
inference_config.json stores the calibrated threshold (0.87) and all  
Optuna best hyperparameters for reproducibility.
```

9.2.2 Flask Web Application

A four-step analysis pipeline: (1) Input ingestion (raw text, file upload as PDF/DOCX/HTML, or LinkedIn URL); (2) LangChain `job_parser_agent` extracts structured fields matching the EMSCAD schema; (3) RoBERTa classification produces fraud probability and binary verdict; (4) 12-tool agentic verification runs in parallel, followed by OpenRouter LLM generating the structured narrative investigation report.

9.2.3 Chrome Extension (MV3)

A Manifest V3 Chrome extension injects a DOM scraper onto LinkedIn job listing pages. When the user clicks 'Analyze Job', the extension scrapes visible job fields, sends them to the Gemini API, and displays a browser overlay showing the fraud probability, key evidence flags, and a brief explanation — without the user leaving the job listing page.

9.3 Agentic Verification Pipeline

The LangChain ReAct agent uses 12 verification tools to cross-check specific claims extracted from the posting:

Tool Category	Tools	Signal Checked
Domain verification	Domain registration check, URL reputation	Company domain age; suspicious TLDs
Contact verification	Email domain / MX record, phone number check	Email domain plausibility; phone format and registration
News and reputation	DuckDuckGo news search	Does the employer appear in any news? Any fraud reports?
Cross-reference	Job board cross-reference	Does this posting appear on multiple legitimate platforms?
Registry	Company registry lookup (stub)	Is the company a registered legal entity?
Document parsing	LangChain document loaders (PDF/DOCX/HTML)	Structured field extraction from uploaded documents

The LLM summary layer (OpenRouter or Gemini) takes all verification results and generates a structured prose investigation report — not a list of numbers or feature weights. This directly addresses the gap in prior explainable AI approaches (LIME/SHAP) that produce outputs opaque to non-technical job seekers.

9.4 System Architecture Overview

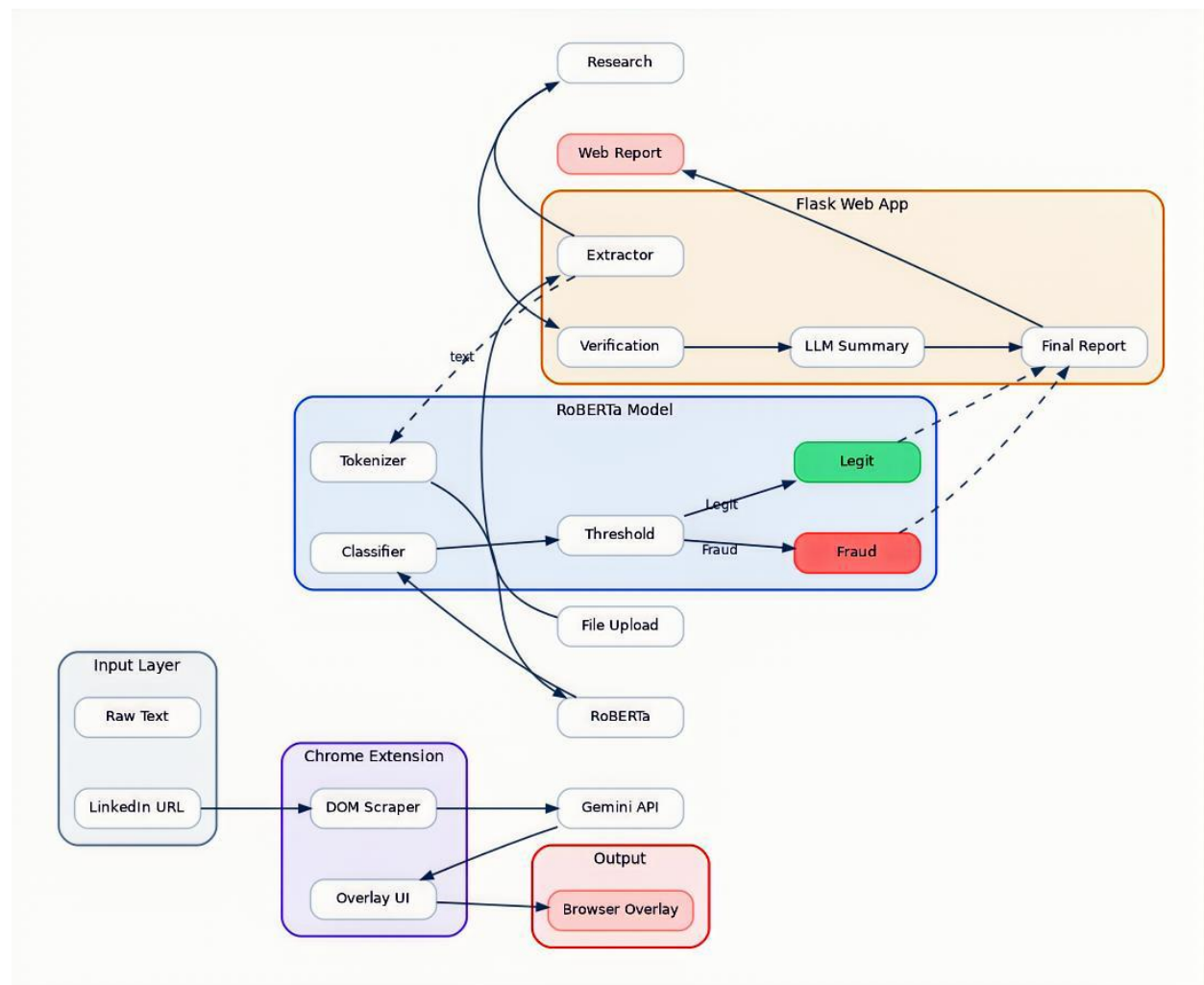


Figure 9: System architecture overview mapping the end-to-end processing pipeline. Left: Input layer handling raw text and LinkedIn URLs via the Chrome Extension DOM Scraper. Center: The RoBERTa model classifier generating probability thresholds. Right: Flask web app orchestrating data extraction, multi-tool verification, and final LLM summary generation.

9.5 Technology Stack

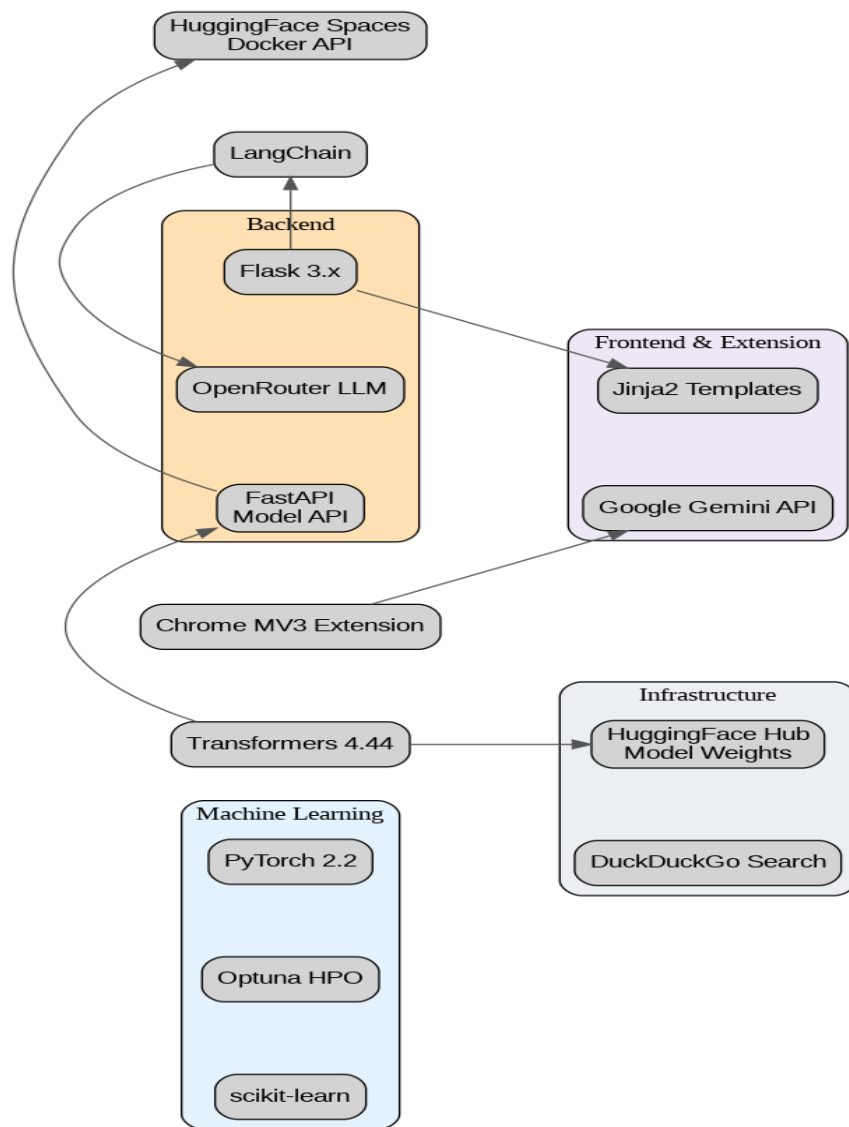


Figure 10: FraudGuard technology stack and deployment dependencies. **Top:** Web infrastructure linking HuggingFace Spaces to the Flask 3.x backend and LangChain agents. **Middle:** Integration with the OpenRouter LLM and frontend interfaces. **Bottom:** Core machine learning frameworks including PyTorch, Optuna, and Scikit-learn driving the inference engine.

9.6 Chrome extension Workflow

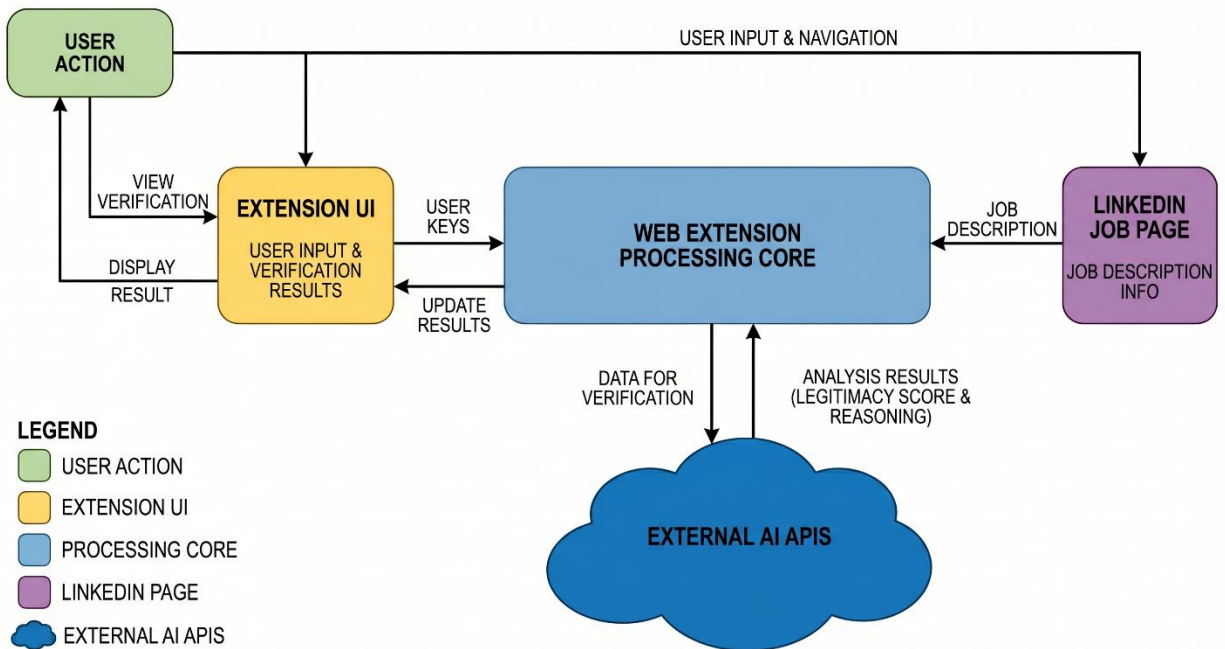


Figure 11: Chrome extension MV3 operational workflow. Left: User action flow and Extension UI managing input and displaying verification results. Center: Web extension processing core mediating data between the page and external AI APIs. Right: LinkedIn job page data extraction logic.

9.7 How It Is Deployed

Component	Platform	Access
RoBERTa model weights	HuggingFace Hub	<code>from_pretrained("aditya963/fraud-job-classifier")</code>
Web-app backend	Local (Flask dev server)	<code>python web-app/app.py</code> → <code>http://localhost:5000</code>
Chrome extension	Browser (load unpacked)	<code>chrome://extensions</code> → Load unpacked → <code>web-extension/</code>

10 Retrospective and Lessons Learned

10.1 What We Would Do Differently

Start HPO with Hard Constraints from Trial 1

The soft-floor versions (v2_1, v3) consumed significant compute without contributing toward the real Mahfouz targets. The v3_1 design — hard precision AND recall floors inside Optuna — should have been the v2_1 design. The key insight that soft floors are gamed by Bayesian optimizers was not obvious before running the experiments, but in retrospect the theoretical reasoning is clear: a Bayesian optimizer is a rational agent that maximizes its objective subject to constraints. Weak constraints produce weak guarantees.

Run Cleanlab Label Noise Audit Before Training

This was planned and documented in Milestone 2 but was never executed. Label noise in 866 fraud training samples — even a small number of mislabeled postings — could explain part of the validation → test gap. It should have run after the first training loop using the saved predicted probabilities.

Test Sliding-Window Attention from the Start

The ~11% truncation rate is a real accuracy penalty. Longformer-style chunking with pooling over segments was identified in the Milestone 3 Architecture Report as the appropriate long-term fix, but was deprioritized in favour of simpler truncation. For a dataset where fraudulent postings are on average shorter than legitimate ones, this asymmetry means truncation disproportionately affects legitimate postings — but some long fraudulent descriptions still lose signal.

Ablate Synthetic Augmentation (v5 synth) Against v3_1

The GPT-4 synthetic augmentation approach has strong theoretical merit — it reduces the 20:1 class imbalance at the data level, complementing the loss-function reweighting. However, it was never rigorously ablated against the v3_1 baseline. The synthetic data quality depends on prompt consistency and generation variance that has not been measured. It remains a promising hypothesis, not a validated improvement.

10.2 Key Lessons

Soft HPO Constraints Are Not Constraints

Putting target floors into Optuna's reporting stage instead of its objective function allows the optimizer to find configurations that satisfy the letter of the constraint while violating its spirit. Hard binary scoring — trial scores 0.0 if any floor is not met — is the correct design when production targets are defined.

LoRA Is a Hypothesis, Not a Default

Parameter-efficient fine-tuning requires sufficient semantic proximity between pretraining and downstream task distributions. With 866 fraud samples in a domain distinctly different from general web text, LoRA's frozen encoder could not shift to fraud-specific pattern recognition. The $F1=0.836$ plateau was not a tuning failure — it was a fundamental capacity constraint.

Threshold Calibration Is a Free Lunch

The training objective optimizes cross-entropy or focal loss; it does not optimize F1 at a specific threshold. Post-training threshold calibration on the validation set is a standard technique that most published papers report but many implementations skip. Moving from 0.5 $\rightarrow$ 0.87 recovered 2–4 F1 points at zero cost. At deployment, the threshold can be adjusted to meet changing operational requirements without retraining.

Hardware Constraints Are Real Constraints

DeBERTa-v3's theoretical superiority on NLP benchmarks does not survive the combination of T4 GPU VRAM limits, focal loss on a highly imbalanced dataset, and a small training set. Theory and practice diverge when training stability matters more than benchmark ranking. The correct conclusion is not that DeBERTa is inferior — it is that architectural complexity interacts poorly with these specific constraints.

Absence of Data Is Data

The decision not to impute missing metadata fields was one of the most impactful preprocessing choices in the project. A posting structurally distinguished by what it lacks — no company profile, no salary range, no company logo — is meaningfully different from one that has these fields. Imputation would have erased this implicit fraud signal.

11 Future Work

11.1 Priority 1 — RoBERTa + DeBERTa Soft-Voting Ensemble

The Milestone 3 Architecture Report we explicitly identified a soft-voting ensemble of RoBERTa-base and DeBERTa-v3-base as the most promising path to meeting all four Mahfouz targets simultaneously. The core insight is that DeBERTa's instability in standalone training — NaN losses, VRAM pressure, slow convergence — is manageable when it serves as a secondary voter alongside a stable RoBERTa anchor.

In a soft-voting ensemble, each model's independent probability estimates are averaged before thresholding. DeBERTa's errors are largely independent of RoBERTa's errors (they arise from different architectural biases), so averaging reduces the variance of the combined estimate. The ensemble's precision-recall curve is superior to either single model. DeBERTa does not need to train stably alone — it only needs to produce calibrated probability estimates that complement RoBERTa's.

11.2 Near-Term Improvements

- Threshold recovery (immediate, zero training cost): Lower threshold $0.87 \rightarrow \sim 0.82$. Phase 6 threshold sweep data shows recall recovers toward 0.89 while remaining above the precision floor.
- Extend Optuna to 35+ trials: Recommended in v3_1 code comments when the F1 target is not met. More trials may find a configuration where validation recall exceeds 0.89 more comfortably, reducing the train→test gap impact.
- Cleanlab label noise audit: Run Confident Learning on the training set using saved predicted probabilities. Even a small number of mislabeled fraud samples in the 866-sample minority class can meaningfully shift decision boundaries.
- Validate v5_synth augmentation against v3_1 baseline: Rigorous ablation of GPT-4 synthetic fraud postings against the v3_1 baseline with identical HPO configuration. The approach has strong theoretical merit but remains unvalidated.

11.3 Longer-Term Directions

- Multilingual support via xlm-roberta-base: Fraud is not an English-only problem. XLM-RoBERTa extends the same architecture to 100 languages without retraining the preprocessing pipeline, enabling coverage of non-English job platforms.
- Sliding-window attention for sequences > 512 tokens: Longformer-style chunking with segment pooling addresses the ~11% truncation penalty. A hierarchical encoder (chunk → pool → classify) is the architecturally clean solution.

- Knowledge distillation (Milestone 1 Phase 6 plan): Train DistilBERT-base as a student from the RoBERTa teacher using soft label supervision. Expected outcome: ~40% size reduction, <3% F1 drop — enables sub-500ms inference without GPU.
- INT8 quantization via ONNX Runtime: Post-training quantization of the final model for CPU deployment. Expected ~4× inference speedup with <1% accuracy loss.
- Real-time model updates: A feedback loop where user-confirmed fraud cases (via the web app or extension) are periodically used to continue fine-tuning the deployed model, reducing distribution shift over time.

12 References

1. Vidros, S., Koliass, C., Kambourakis, G., & Maglaras, L. (2017). Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset.
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
3. Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.
4. Amaar, A., et al. (2022). Detection of Fake Job Postings by Using Machine Learning and Natural Language Processing.
5. Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection.

13 Appendix

13.1 Sample Model Inference

A. Sample Model Inference

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer
import torch

model = AutoModelForSequenceClassification.from_pretrained("aditya963/fraud-job-classifier")
tokenizer = AutoTokenizer.from_pretrained("aditya963/fraud-job-classifier")
model.eval()

fraud_posting = {
    "title": "Work From Home Data Entry Specialist",
    "description": "Earn $500/day. No experience needed. Send bank details.",
    "company_profile": "",
    "location": "Remote",
    "salary_range": "500-1000",
    "has_company_logo": 0,
}

text = " [SEP] ".join([f"{k}: {v}" for k, v in fraud_posting.items() if v])
inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512, padding="max_length")
with torch.no_grad():
    prob = torch.softmax(model(**inputs).logits, dim=-1)[0][1].item()

print(f"Fraud probability: {prob:.4f}") # -> ~0.92
print("Prediction:", "FRAUDULENT" if prob >= 0.87 else "LEGITIMATE")
```

Figure 12: Sample Python model inference script. The snippet demonstrates loading the fine-tuned sequence classification model and tokenizer from the HuggingFace Hub repository `aditya963/fraud-job-classifier`, structuring a mock fraudulent job posting, and executing a forward pass to print the predictive probability.

13.2 Team Contributions

Team Member	Contribution
Arun Dutta	Literature Review, Gap Analysis Data preprocessing, split strategy, project documentation Pipeline verification, EDA, documentation HPO analysis, training report Results documentation, results synthesis Final report, contribution summary ~25%
Hritik Roshan Maurya	Problem framing, system architecture design LangChain agent (<code>job_parser_agent.py</code>), structured field extraction CLI Model design, Training strategy review Model validation, system integration testing LangChain ReAct agent, real-world PDF/HTML testing API documentation, deployment documentation ~25%
Vivek Bajaj	Dataset identification, DL pipeline design Primary dataset curation, baseline classifier, HuggingFace Hub deployment Training pipeline, Focal Loss implementation Optuna HPO (25 trials), threshold calibration, final eval Threshold tuning (0.87), final test scripts, model artifacts Technical documentation, architecture review ~25%